

METHODOLOGICAL MANUSCRIPT

A Reproducible Median-Spectrum Deep-Learning Pipeline for Multiclass Organ Classification from Hyperspectral Data

*Subject-separated validation, fixed-width wavelength selection,
imbalance handling, and auditable HTC integration*

[AUTHOR NAMES]

[AFFILIATIONS AND CORRESPONDING AUTHOR]

Draft derived from the supplied merged DKFZ pipeline and immutable HTC source tree • 7 August 2026

Abstract

Hyperspectral imaging produces dense spectral measurements that may support objective tissue recognition, but methodological comparisons are easily confounded by information leakage, inconsistent preprocessing, changing model capacity, and incomplete provenance. We describe a configuration-driven pipeline for classifying organ-specific median spectra with the Hyperspectral Tissue Classification (HTC) median-pixel network. The software discovers precomputed median-spectrum comma-separated value files linked to configured HyperGUI and labelling artifacts, validates a common wavelength grid, and assigns complete subjects—not individual spectra—to training, validation, and testing partitions. The active reference configuration targets six rat-organ classes, 100 channels spanning 500–995 nm, and a 60%/20%/20% subject-level split. It additionally defines an external nine-class pig-organ test set. Selective-band experiments preserve a constant 100-channel model input: selected channels are standardized using training-only statistics and scattered back to their original positions, whereas unselected channels are zero-masked. The classifier is a one-dimensional convolutional network with three convolution–batch-normalization–exponential-linear-unit–average-pooling blocks, two fully connected layers, dropout, and a multiclass output head. Training uses Adam, exponential learning-rate decay, cross-entropy loss, deterministic seeding, validation-based checkpoint selection, and balanced oversampling. The pipeline exports manifests, split workbooks, predictions, raw and row-normalized confusion matrices, accuracy, balanced accuracy, macro recall, macro F1, class-specific precision, recall and F1, worst-class recall, software versions, source hashes, and fully resolved parameters. This design provides an auditable basis for wavelength-ablation studies and median-spectrum organ classification while explicitly preventing subject leakage and model-capacity drift.

Keywords: hyperspectral imaging; median spectrum; organ classification; deep learning; wavelength selection; reproducibility; subject-level splitting.

1. Introduction

Hyperspectral imaging (HSI) measures tissue reflectance across contiguous wavelength bands and can expose biochemical and physiological differences that are not apparent in conventional color images. In supervised tissue classification, however, a high apparent accuracy may arise from experimental design choices rather than generalizable biological signal. Spectra from the same subject are correlated; placing them in multiple partitions leaks subject-specific information. Band-selection experiments can also become unfair when a narrower input changes the network architecture or parameter count. Finally, code assembled from multiple research repositories can be difficult to audit unless the exact configuration and source versions are retained.

The supplied software addresses these problems through a cross-platform orchestration layer around the unmodified HTC median-pixel implementation. The intended analytical unit is one median spectrum associated with an acquisition and organ label. A strict, reusable YAML configuration controls source discovery, wavelength selection, splitting, training, imbalance handling, and evaluation. The principal contribution is methodological integration: all experimental variants use one subject split, the same fixed-width network input, training-only normalization, validation-selected checkpoints, and machine-readable provenance.

This manuscript describes the implemented workflow, specifies the reference experiment encoded in the repository, and defines a reporting framework for subsequent empirical results. Because the supplied repository contains no completed scenario outputs or clinical acquisition protocol, performance values, cohort sizes, acquisition details, run time and ethics statements cannot be inferred and remain to be inserted from the study record.

2. Materials and Methods

2.1 Study design and analytical objective

The pipeline implements supervised multiclass classification of organ-specific median reflectance spectra. Each row of an internal manifest corresponds to one spectrum file and includes the file path, class label, integer label index, subject identifier, acquisition timestamp, image identifier and annotation name. The active internal reference configuration defines six rat-organ classes—stomach, small bowel, colon, liver, pancreas and kidney—and the external evaluation configuration defines nine pig-organ classes—stomach, small bowel, colon, liver, pancreas, kidney, spleen, bladder and omentum. External labels define evaluation-matrix rows and are not supplied to the trained model loss. The biological source, number of subjects and acquisitions, inclusion and exclusion criteria, and reference-standard annotation procedure must be supplied from the experimental protocol before submission.

2.2 Software architecture and configuration

A single YAML file is the public control surface. Platform-specific paths select an authoritative HTC source tree without modifying it, while scenario-relative folders hold all generated artifacts. Configuration validation requires consistent organ keys across experiment folders, labelling-file patterns, and HyperGUI-folder patterns; positive split ratios summing to one; required training parameters; legal wavelength limits; and exactly one imbalance strategy. Completed scenario folders are protected by a run-parameter marker, preventing accidental overwrite of a prior run.

Component	Reference setting	Methodological role
Spectral input	100 channels, 500–995 nm; all bands selected	Common grid for discovery and wavelength variants
Internal classes	6 rat-organ labels	Stomach, small bowel, colon, liver, pancreas and kidney
External classes	9 pig-organ labels	Independent evaluation; 9 true-class rows × 6 output columns
Partitioning	60% train / 20% validation / 20% test	Subject-level; seed 42; ≤20,000 candidate assignments
Training	≤150 epochs; batch 64; learning rate 2×10^{-4}	One device; accelerator auto; 32-true precision; 0 workers
Epoch size	Number of training spectra at run time	Replacement sampling under balanced oversampling
Normalization	Per-channel z-standardization	Mean and population SD estimated from training spectra only
Imbalance	Balanced oversampling	Mutually exclusive with class-weighted loss
Checkpoint	Maximum validation accuracy	Best checkpoint used for validation prediction and final testing
Summary metrics	Accuracy; balanced accuracy/macro recall; macro F1	Plus row count, class counts, worst class and worst-class recall
Class metrics	Support; correct; precision; recall; F1	Plus dominant confused class and confusion count

Component	Reference setting	Methodological role
Outputs	CSV, Excel, PNG, PDF and JSON; 300 dpi	Predictions, matrices, metrics, splits and provenance

2.3 Data discovery and eligibility

For every configured organ, the software recursively identifies HyperGUI directories matching the specified exact name or glob pattern. A recording is eligible only when a configured labelling text file is found beside the selected HyperGUI directory and a spectrum CSV matches one of the configured filename patterns. HyperGUI folders without the required labelling file are ignored and reported. Duplicate file paths are removed, and a spectrum assigned to more than one class raises an error. At least two classes must remain after filtering.

Spectrum readers accept either one numeric intensity column or at least two numeric columns, in which case the second column is treated as the spectral value and the first may define wavelength. Spectra must contain the configured number of channels. Subject identifiers are parsed from path components matching the expected animal/experiment naming conventions; failure to identify a subject stops preparation. Timestamps are similarly obtained from acquisition-folder names. These identifiers are combined with the spectrum stem to create a stable image name used by HTC.

2.4 Wavelength-grid validation and selective-band representation

The configured spectral domain is 500–995 nm. When a two-column file supplies a strictly increasing wavelength vector, that vector is used; otherwise the grid is inferred by evenly spacing the configured minimum and maximum across the expected channel count. Every spectrum must resolve to the same available grid and selected indices. Selective input may combine exact wavelengths and inclusive ranges. An exact wavelength must lie on the grid within a tolerance equal to 1% of the median band spacing (minimum 10^{-6}); a range must contain at least one available band.

To isolate information content from network capacity, selective-band experiments retain the original 100-position input. Training-derived means and standard deviations are calculated only for selected bands. Standardized values are then scattered into their original wavelength positions in a zero-initialized vector, and all unselected positions remain zero after transforms. Thus every variant has identical tensor width and model parameterization.

2.5 Subject-level partitioning

Subjects, rather than spectra, are the indivisible units of partitioning. The configured target fractions are 0.60 for training, 0.20 for validation and 0.20 for testing. Integer partition sizes are obtained by rounding the validation and test targets, with at least one subject assigned to each, and assigning the remainder to training. Before assignment, each class must occur in at least as many distinct subjects as the number of partitions required to contain all classes. Using split seed 42, the algorithm evaluates as many as 20,000 random subject permutations. Candidates are ranked first by the number of missing class-partition combinations and subsequently by the sum, over partitions, of the coefficient of variation of class-specific spectrum counts. The lowest-scoring feasible assignment is retained. No subject can occur in more than one partition, and all six internal classes are required in training, validation and testing. When external testing is enabled, the external cohort replaces the internal test manifest; it remains independent of model fitting and may contain a different class set.

The selected assignment is written to split-specific manifests and per-organ Excel workbooks. A summary JSON records the seed, target ratios, subject lists, row counts, and class counts. The complete manifest is protected by a SHA-256 digest.

2.6 Spectral preprocessing

The file patterns target precomputed masked, zero-derivative median spectra. The pipeline does not recreate pixel masks or median aggregation; those upstream steps therefore require independent documentation from the acquisition and annotation protocol. During model preparation, selected channels are standardized using the mean and population standard deviation calculated across training spectra. A zero standard deviation is replaced by one. The same training statistics are applied unchanged to validation and test spectra. No validation or test observations contribute to preprocessing parameters.

2.7 Neural-network architecture

The classifier reuses the HTC ModelPixel architecture with a single spectral input channel. Three valid one-dimensional convolutions use kernel size 5 and output 64, 32 and 16 feature maps, respectively. Each convolution is followed by one-dimensional batch normalization, an exponential linear unit activation and average pooling with kernel size 2. The convolutional representation is flattened and adaptively reduced to the dimension determined for the configured input width. Two dense layers contain 100 and 50 units, each followed by batch normalization, exponential linear unit activation and dropout with probability 0.2. A task-specific classification head maps the 50-dimensional representation to six logits in the active internal experiment. External evaluation retains these six output columns even

when the external matrix contains nine true-class rows. This architecture is inherited directly from the authoritative HTC runtime.

2.8 Model training and imbalance handling

The integration layer does not implement the `gpu_smoke`, `gpu_practical` or `gpu_paper` profiles shown in the preliminary notes; instead, all reported settings are read directly from the active YAML configuration and saved again in the resolved run parameters. Training proceeds for at most 150 epochs using Adam with learning rate 2×10^{-4} and zero weight decay, followed by an ExponentialLR scheduler with multiplicative factor 0.9. The batch size is 64, numerical precision is 32-bit (32-true), and one device is selected with the accelerator set to auto. The number of data-loader workers is zero. Randomness is initialized with seed 42, including worker initialization. The training epoch size is not a fixed numerical constant in the YAML; it resolves at run time to the number of training spectra. With balanced oversampling, this number of observations is sampled with replacement in each epoch. The objective is multiclass cross-entropy loss.

Exactly one of three imbalance policies is permitted. With no adjustment, a replacement random sampler draws an epoch equal in length to the training set. With balanced oversampling, each observation receives sampling weight inversely proportional to the frequency of its class and is drawn with replacement. With class weighting, the HTC loss applies one of four configured weight transformations (inverse-frequency, balanced, soft-minimum, or negative-log-likelihood form). Oversampling and class weighting cannot be enabled simultaneously. The supplied reference configuration uses balanced oversampling.

2.9 Model selection and evaluation

At every epoch, HTC aggregates validation confusion matrices by subject and logs mean subject-level accuracy. The checkpoint attaining maximum validation accuracy is retained, together with the most recent checkpoint. After fitting, the best checkpoint is used to generate row-level validation predictions and to evaluate the held-out test loader. Class predictions are obtained as the argmax of the model logits. For external evaluation, prediction is performed without calculating an HTC test loss because external class identifiers need not correspond one-to-one to the training targets.

For both validation and test partitions, the pipeline calculates a count confusion matrix C and its row-normalized form, $C_{ij}/\sum_j C_{ij}$. Matrix rows denote true classes and columns denote trained-model classes; consequently, the active external experiment yields a 9×6 matrix. Let N be the number of evaluated spectra, $TP_i = C_{ii}$, $n_i = \sum_j C_{ij}$ and $p_i = \sum_j C_{ji}$ for a class name present in both axes. Overall accuracy is $\sum_i TP_i/N$. Class-specific precision is TP_i/p_i , recall (sensitivity) is TP_i/n_i and F1 is $2 \cdot \text{precision}_i \cdot \text{recall}_i / (\text{precision}_i + \text{recall}_i)$; a zero denominator yields zero. For an external-only class with no

matching model-output column, TP_i , precision, recall and F1 are defined as zero. Balanced accuracy and macro recall are identical in this implementation and equal the unweighted mean class recall, whereas macro F1 is the unweighted mean class F1. The output further records N , the number of true classes, the number of predicted classes, the class with minimum recall and that worst-class recall; alphabetical order breaks a tie in minimum recall. Per-class output additionally reports label index, support n_i , correct count TP_i , the most frequently confused predicted class and its count. All metrics are spectrum-level because each manifest row is one median spectrum. Specificity, negative predictive value, micro- or weighted-average scores, receiver-operating-characteristic area under the curve, calibration indices, confidence intervals and subject-level aggregate metrics are not calculated by the current code and therefore are not reported as implemented outcomes.

2.10 Stepwise wavelength analysis

Optional forward and reverse analyses quantify how performance changes as contiguous spectral coverage expands. Forward runs begin with the first two eligible bands at the lower boundary and add one channel per run until the configured forward stop; reverse runs begin with the last two eligible bands and expand downward until the reverse stop. Both modes are disabled in the active reference configuration, whose stop values are 995 and 500 nm, respectively. Discovery and subject splitting occur once at the master level and are reused by every child run. Each child receives a generated YAML configuration and a fixed-width selected-band representation. Completed children are recognized from their run-parameter record and skipped on restart. Aggregate tables contain validation and test accuracy together with selected-channel count and boundary wavelength, and the corresponding accuracy trajectories are plotted.

2.11 Reproducibility and provenance

Each scenario saves the source YAML, resolved configuration, resolved HTC configuration, selected indices and wavelengths, split manifests, best checkpoint path and score, Python/platform/PyTorch/Lightning versions, imported HTC module path, and SHA-256 fingerprints of critical HTC files. The integration repository is intentionally separate from the HTC runtime, which is placed first on the Python import path but treated as read-only. A baseline manifest records hashes for the default model configuration, median-pixel dataset and Lightning module, common dataset and training infrastructure, configuration utility, and training entry point. This separation permits investigators to verify that an analysis used the intended upstream implementation.

2.12 Software verification

The merged project includes automated tests for configuration validation, subject splitting, wavelength parsing/masking, evaluation metrics, and stepwise run construction. Prior to publication, the complete

test suite should be executed in the pinned analysis environment and its pass/fail report archived. A prospective end-to-end verification should additionally confirm that (i) split subjects are disjoint, (ii) every required class occurs in every partition, (iii) preprocessing statistics originate only from training data, (iv) selected-band variants have equal parameter counts, (v) checkpoint selection uses validation data only, and (vi) exported predictions reproduce all reported metrics.

3. Technical Validation and Reporting Plan

The current code package contains the analysis implementation but no completed scenario outputs; accordingly, no empirical performance estimate can be stated from the repository alone. A completed report should reproduce all metrics written by the software: validation and test accuracy, balanced accuracy (macro recall), macro F1, evaluated-spectrum count, true- and predicted-class counts, worst class and worst-class recall, as well as class-specific support, correct count, precision, recall, F1 and dominant confusion. It should also report subjects and spectra by class and partition, the selected wavelength grid, training duration and hardware, the validation-selected epoch, and raw and row-normalized confusion matrices. Confidence intervals or subject-level distributions require an additional prespecified analysis because the present implementation does not compute them. Wavelength experiments should present validation and test trajectories separately and identify any selection made after inspecting test performance as exploratory.

Required item	Source artifact	Manuscript destination
Cohort and class counts	data/split_summary.json; workbooks	split Participants/data subsection and Table 1
Exact spectral selection	data/wavelengths.json	Preprocessing subsection
Best model and environment	run_config/run_parameters.json	Training and reproducibility subsections
Validation/test summary	validation_metrics.csv; testing_metrics.csv	Results table
Class-specific performance	*/csv/per_class_metrics.csv	Results and supplement
Error structure	raw and normalized confusion matrices	Main or supplementary figure
Stepwise trajectories	stepwise results/*.csv and plots	Wavelength-ablation results

4. Discussion

The implemented workflow makes several design choices that strengthen internal validity. Subject-separated partitions address the strongest leakage risk in repeated spectral acquisitions. Training-only

standardization prevents distributional information from validation or testing from entering the fitted representation. Fixed-width masking holds architecture and parameter count constant across band selections. A single master split improves comparability among stepwise experiments, while strict configuration checks and extensive run metadata make deviations visible.

These strengths do not remove limitations inherent to the current implementation. First, the pipeline consumes already aggregated spectra and therefore cannot audit pixel-level segmentation, masking quality, or the calculation of organ medians. Second, reported confusion-matrix metrics are row-level and may overweight subjects with more acquisitions. Third, the random-search split optimizer prioritizes class coverage and class-count balance but is not a formal stratified group optimization procedure. Fourth, test accuracy is exported for every stepwise child; choosing a wavelength subset on those values would leak test information. Model selection should use validation trajectories, with test results reserved for a single prespecified comparison. Fifth, dependency versions are recorded at runtime but are not pinned in the lightweight merged-project requirements file; environment locking or a container is recommended.

External validity will depend on acquisition hardware, imaging conditions, species or patient population, annotation practice, and institutional setting, none of which can be reconstructed reliably from the supplied code. A final submission should describe the HSI camera, calibration and illumination, acquisition distance and angle, tissue handling, operator training, exclusion criteria, blinding, ethical approval, and data availability. Independent-camera or independent-center evaluation would provide a stronger assessment of robustness than a random subject split from one source population.

5. Conclusion

This pipeline provides a transparent and reusable method for median-spectrum organ classification and wavelength-ablation analysis using the HTC median-pixel network. Its defining safeguards are subject-level partitioning, training-only standardization, constant-width selective-band inputs, mutually exclusive imbalance strategies, validation-based checkpointing, and comprehensive provenance. Once completed run artifacts and study-level metadata are inserted, the framework can support a fully reproducible empirical manuscript without conflating software capability with unobserved performance.

Declarations

Ethics approval. [INSERT approving body, protocol identifier, consent/animal-welfare statement, and relevant reporting guideline.]

Consent for publication. [INSERT OR STATE NOT APPLICABLE.]

Data availability. [INSERT dataset access conditions, repository/DOI, and restrictions.]

Code availability. The manuscript was derived from the supplied “dkfz final code” integration project and the HTC source tree. Insert the public repository URL, release tag/commit, license, and archival DOI before submission.

Funding. [INSERT FUNDING SOURCES AND GRANT NUMBERS.]

Competing interests. [INSERT DECLARATION.]

Author contributions. [INSERT CRediT CONTRIBUTIONS.]

References

1. Sellner J, Seidlitz S. Hyperspectral Tissue Classification (HTC) [software]. Zenodo. doi:10.5281/zenodo.6577614. Verify version and access date for the final analysis.
2. Seidlitz S, Sellner J, Odenthal J, et al. Robust deep learning-based semantic organ segmentation in hyperspectral images. *Medical Image Analysis*. 2022;80:102488. doi:10.1016/j.media.2022.102488.
3. Paszke A, Gross S, Massa F, et al. PyTorch: an imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*. 2019;32.
4. Kingma DP, Ba J. Adam: a method for stochastic optimization. *International Conference on Learning Representations*. 2015.
5. Clevert D-A, Unterthiner T, Hochreiter S. Fast and accurate deep network learning by exponential linear units (ELUs). *International Conference on Learning Representations*. 2016.
6. [ADD primary references for the imaging system, biological dataset, annotation protocol, and any public data release used in the final study.]

Appendix A. Reproducible execution

After adapting paths and preserving a copy of the final YAML, validate configuration, prepare the manifests, and run the complete scenario with the project command-line interface. The completed scenario directory—not only figures—should be archived. It contains the source and resolved configurations, manifests and split details, checkpoints and logs, validation and test predictions, matrices, metrics, software versions, and source fingerprints.

```
python -m median_pipeline validate --config <scenario>/manifold_settings.yaml
python -m median_pipeline prepare --config <scenario>/manifold_settings.yaml
python -m median_pipeline run --config <scenario>/manifold_settings.yaml
```

Appendix B. Items requiring investigator confirmation

Authorship and outlet: Author list, affiliations, corresponding author, and target journal.

Study population: Species/patient population, acquisition sites and dates, sample-size rationale, and exclusions.

Imaging protocol: Camera model, illumination, calibration, acquisition geometry, and raw-to-spectrum processing.

Reference standard: Ground-truth annotation protocol, annotator expertise, blinding, and quality control.

Governance: Ethics approval and consent or animal-welfare details.

Empirical results: Final counts, hardware, software versions, runtime, uncertainty analysis, and numerical results.

Archiving: Repository URL, commit/release, license, archival DOI, and data-access statement.